SUMMER TRAINING PROJECT REPORT

Smart Blood Donation Emergency Matching System

This is Submitted By:

Abhin S

Reg No. 12411257

Group ID : 52

Section-324EQ

Under the Guidance of

Ravi Kant Sahu

School of Computing & Artificial Intelligence
Lovely Professional University, Phagwara

DECLARATION

I hereby certify that the project titled "**Smart Blood Donation Emergency Matching System**" is a genuine representation of my work, conducted as part of the Project requirement for the CA-PETV143 at Lovely Professional University, Phagwara. This work was carried out under the supervision of Professor Dr. Ravi Kant Sahu during the period of June to July 2026. All the data and insights presented in this project report are derived from my dedicated efforts and are entirely original.

Name of Student : Abhin .S
Registration No.:12411257
Group : 52

CERTIFICATE



CENTRE FOR
PROFESSIONAL
ENHANCEMENT

Certificate No. 492215

NAAC GRADE A++

Certificate of Merit

This is to certify that Mr./Ms. **Abhin S** S/D/W/o **Mr. Santhosh Kumar B**

student of **School of Computing and Artificial Intelligence** Registration No. **12411257**

pursuing **Bachelor of Technology (Computer Science and Engineering - Artificial Intelligence and Machine Learning)** completed

skill development course named **Fundamentals of Data Structures: Learn, Apply and Build Projects**

organized by **Centre for Professional Enhancement** Lovely Professional University

from **11-06-2026** to **25-07-2026** and obtained **B** Grade.

Date of Issue: 13-08-2026
Place of Issue: Phagwara (India)

Prepared by
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of Computing and Artificial Intelligence

ABSTRACT

Blood donation plays a vital role in saving human lives during medical emergencies, surgeries, and accidents. However, one of the major challenges faced by hospitals and blood banks is finding suitable blood donors quickly when blood is urgently required. Traditional methods of maintaining donor records are often manual, time-consuming, and inefficient, making it difficult to search for the right donor during critical situations. To address this problem, the Smart Blood Donation Emergency Matching System has been developed as a console-based application using C++.

The primary objective of this project is to provide a simple and efficient system for managing blood donor information and handling emergency blood requests. The application allows users to register new donors, update donor details, search donors based on blood groups, maintain blood request records, and keep track of donation history. It also provides basic reports such as the total number of registered donors, available donors, and blood group-wise donor statistics.

The project has been developed using the principles of Object-Oriented Programming (OOP) and makes effective use of Data Structures to improve the efficiency of different operations. A vector is used to store donor records dynamically, an unordered_map enables faster searching of donors based on blood groups, a priority_queue manages emergency blood requests by giving higher priority to urgent cases, and a list is used to maintain donation history. File handling is implemented to store donor records and request information permanently, ensuring that data is retained even after the application is closed.

This project demonstrates the practical application of Data Structures and Algorithms in solving a real-world social problem. It also enhances programming skills, problem-solving abilities, and understanding of Object-Oriented Programming concepts. Although the system is designed as a college-level console application, it provides a strong foundation for future enhancements such as database integration, graphical user interfaces, and hospital management system connectivity.

.

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
CHAPTER NO.	TITLE	PAGE NO.
1	INTRODUCTION	5
1.1	Problem Statement	5
1.2	Objective	5
1.3	Project Module	6
1.4	Technology Used	8
2	Module Description	8
2.1	Donor Management	8
2.2	Blood Request Management	8
2.3	Emergency Matching	9
2.4	Donation History	10
2.5	Reports	10
3	Implementation	12
3.1	Project Workflow	12
3.2	Data Structures Used	12
3.3	Important Source Code with Explanation	13
4	Screenshots of the Project	17
5	Future Scope	21
6	Conclusion	22
7	References	

CHAPTER 2

INTRODUCTION

1.1 Problem Statement

Blood is one of the most important resources in the healthcare system. During accidents, surgeries, medical emergencies, and treatments, patients often require blood immediately. One of the major challenges faced by hospitals and blood banks is finding suitable blood donors in a short period of time. In many situations, donor information is maintained manually in registers or scattered across different records, making the search process slow and inefficient. This delay can become critical during emergency situations where every minute is important.

The **Smart Blood Donation Emergency Matching System** is developed to solve this problem by maintaining donor information in a structured digital format. The system allows users to register donors, search donors based on blood groups, manage emergency blood requests, and maintain donation history. It uses Data Structures and Object-Oriented Programming concepts to perform operations efficiently.

The project is designed as a stand-alone console application in C++, making it suitable for educational purposes while demonstrating the practical use of Data Structures in solving a real-world social problem.

1.2 Objectives

The main objective of this project is to develop a simple and efficient blood donation management system that demonstrates the practical implementation of Data Structures using C++.

The objectives of the project are:

- To maintain donor information digitally.
- To register new blood donors.
- To search donors quickly based on blood groups.
- To manage emergency blood requests.
- To maintain donation history.
- To reduce the time required to find suitable donors.
- To implement Object-Oriented Programming concepts.
- To demonstrate the practical use of Data Structures.
- To develop a user-friendly console application.
- To improve programming and problem-solving skills.

1.3 Project Modules

The project is divided into different functional modules to make the application organized and easy to maintain.

Module 1 – Donor Management

This module manages all donor-related information. It allows users to register new donors, update donor details, search donors, delete donor records, and display all registered donors.

Main Functions:

- Register Donor
- View All Donors
- Search Donor
- Update Donor
- Delete Donor

Module 2 – Blood Request Management

This module is responsible for maintaining blood requests received from hospitals or patients. The user can add new blood requests and view all existing requests.

Main Functions:

- Add Blood Request
- View Blood Requests

Module 3 – Emergency Matching

This module searches for suitable donors whenever an emergency blood request is received. The search is performed using the blood group entered by the user. The system displays available donors matching the requested blood group.

Main Functions:

- Search donor by blood group
- Display available donor
- Match donor with request

Module 4 – Donation History

This module stores the details of successful blood donations. It helps maintain a history of donations made through the system.

Main Functions:

- Store donation details
- Display donation history

Module 5 – Reports

This module generates simple reports related to donors and blood requests.

Main Functions:

- Total registered donors
- Available donors
- Blood group-wise donor count

1.4 Technologies Used

Technology	Purpose
C++	Main programming language used to develop the application.
Object-Oriented Programming (OOP)	Used to organize the project into different classes and improve code reusability.
Standard Template Library (STL)	Used to implement Data Structures such as vector, unordered_map, priority_queue, and list.
Visual Studio Code	Code editor used for writing and managing the project.
MinGW g++ Compiler	Used to compile and execute the C++ project.
File Handling (fstream)	Used to permanently store donor records, blood requests, and donation history.

CHAPTER 2

MODULE DESCRIPTION

2.1 Donor Management

Description

The Donor Management module is the core module of the Smart Blood Donation Emergency Matching System. It is responsible for storing and managing all blood donor information in the system. This module allows the user to register new donors, view all registered donors, search donors using different criteria, update donor information whenever required, and delete donor records if they are no longer needed.

When a new donor is registered, the system collects important details such as the donor's name, age, gender, blood group, phone number, city, and availability status. Each donor is assigned a unique Donor ID, making it easier to identify and manage records.

The system also provides search functionality to quickly locate donors based on their Donor ID or blood group. The update feature allows modification of donor information if contact details or availability change, while the delete feature removes donor records from the system. All donor information is stored in memory during execution and saved to a file so that it remains available even after the program is closed.

This module demonstrates the practical use of **Object-Oriented Programming** by representing each donor as an object of the Donor class. The donor records are stored using a **vector**, which allows dynamic insertion and deletion of records.

Main Functionalities

- Register a new donor.
- View all registered donors.
- Search donor by Donor ID.
- Search donor by blood group.
- Update donor details.
- Delete donor record.
- Save donor information.

2.2 Blood Request Management

Description

The Blood Request Management module is responsible for recording and managing blood requests received from hospitals or patients. Whenever blood is required, the user can create a new request by entering the patient's name, required blood group, and priority level.

Each request is assigned a unique Request ID for easy identification. The system stores all requests and allows users to view them whenever required. This helps maintain a proper record of pending and completed blood requests.

The module also supports emergency requests by assigning higher priority to urgent cases. This ensures that critical requests are processed before normal requests. The request information is stored permanently using file handling so that previous records are not lost.

This module demonstrates how real-life emergency requests can be organized efficiently using appropriate data structures.

Main Functionalities

- Add a new blood request.
- Store patient details.
- Assign priority to requests.
- View all requests.
- Save request information.

Screenshot: *(Insert screenshot of Blood Request Module)*

2.3 Emergency Matching

Description

The Emergency Matching module is designed to quickly identify suitable blood donors for emergency situations. When a blood request is received, the system searches the available donor records and displays donors who have the required blood group.

To improve search efficiency, an **unordered_map** is used to organize donor records according to their blood groups. This reduces the time required to search compared to checking every donor one by one.

For emergency requests, a **priority_queue** is used so that requests marked as "Emergency" are processed before normal requests. This simulates how hospitals prioritize critical patients during emergencies.

Once a suitable donor is found, the system displays the donor details so that the patient or hospital can contact the donor quickly.

Main Functionalities

- Search donor by blood group.
- Match donor with patient request.
- Handle emergency requests first.
- Display available donor details.

Screenshot: *(Insert screenshot of Emergency Matching)*

2.4 Donation History

Description

The Donation History module maintains a record of successful blood donations. Whenever a donor successfully donates blood, the transaction is recorded in the system along with important details such as the donor ID, donor name, patient name, blood group, and donation date.

A **list** data structure is used to store the donation history because it allows easy insertion of new donation records while maintaining the order in which donations occurred.

Maintaining donation history helps users keep track of previous donations and provides useful information for future reference.

Main Functionalities

- Store donation records.
- Display donation history.
- Maintain chronological order.
- Save donation history using file handling.

Screenshot: *(Insert screenshot of Donation History)*

2.5 Reports

Description

The Reports module provides a summary of the data stored in the system. Instead of viewing individual records, users can quickly obtain useful information through different reports. These reports help in analyzing donor availability and understanding the distribution of different blood groups.

The module displays the total number of registered donors, the number of available donors, and the count of donors belonging to each blood group. These reports make it easier to monitor the overall status of the blood donation system.

The report generation process uses the stored donor information and performs simple calculations to produce meaningful summaries.

Main Functionalities

- Display total registered donors.
- Display available donors.
- Show blood group-wise donor statistics.
- Generate summary reports.

CHAPTER 3

IMPLEMENTATION (SOURCE CODE OF THE PROJECT)

3.1 Introduction

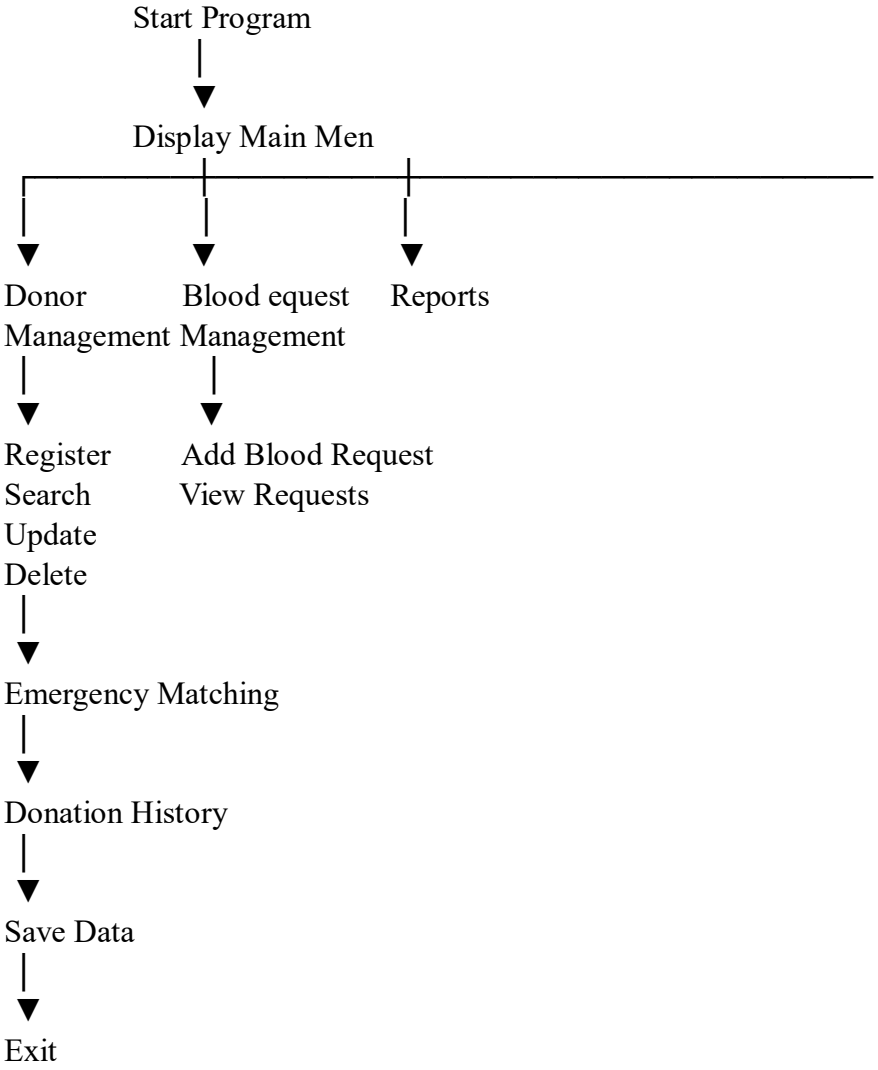
The Smart Blood Donation Emergency Matching System has been developed using **C++** by applying the concepts of **Object-Oriented Programming (OOP)**, **Standard Template Library (STL)**, and **File Handling**. The project is implemented as a console-based application and is divided into different modules to improve readability, maintainability, and code reusability.

The implementation focuses on efficiently managing donor information, processing blood requests, handling emergency cases, and maintaining donation records. Appropriate data structures have been selected based on the requirements of each module to improve performance while keeping the project simple and easy to understand.

The project is organized into multiple source and header files, where each file is responsible for a specific functionality such as donor management, request management, file operations, and the main menu.

3.2 Project Workflow

The overall workflow of the system is shown below.



This workflow represents the sequence of operations performed by the application. The user interacts with the menu-driven interface and selects the required module. Each module performs its assigned task and returns to the main menu after completion.

3.3 Data Structures Used

One of the major objectives of this project is to demonstrate the practical implementation of Data Structures. Different data structures have been selected based on the nature of the operations performed in the system.

3.3.1 Vector

Purpose

A **vector** is used to store all donor records dynamically.

vector<Donor> donors;

Why Vector?

The number of blood donors is not fixed. New donors can register at any time, and existing donor records may also be deleted or updated. A vector allows dynamic memory allocation, making it suitable for storing a changing number of donor records.

Operations Performed

- Add new donor
- Display all donors
- Update donor
- Delete donor
- Traverse donor records

Time Complexity

Operation	Complexity
Insertion (End)	O(1) (Average)
Search	O(n)
Update	O(n)
Delete	O(n)

Why Not Array?

An array has a fixed size, whereas a vector automatically grows or shrinks as donor records are added or removed. Therefore, a vector provides greater flexibility for this application.

3.3.2 Unordered Map

Purpose

An **unordered_map** is used to organize donor records based on blood groups for faster searching.

unordered_map<string, vector<int>> bloodIndex;

Why Unordered Map?

Instead of checking every donor record one by one, the unordered map directly groups donors according to their blood groups. This significantly reduces the search time when looking for donors with a specific blood type.

Operations Performed

- Search donor by blood group

- Store donor indexes
- Update blood group mapping

Time Complexity

Operation	Complexity
Insert	O(1) Average
Search	O(1) Average
Delete	O(1) Average

Advantages

- Faster searching
- Easy grouping of blood donors
- Efficient implementation

3.3.3 Priority Queue

Purpose

A **priority_queue** is used to manage emergency blood requests.

priority_queue<BloodRequest> emergencyQueue;

Why Priority Queue?

Emergency cases should always be handled before normal requests. A priority queue automatically places higher-priority requests at the front, ensuring that urgent cases are processed first.

Operations Performed

- Add emergency request
- Process highest-priority request
- Remove completed request

Time Complexity

Operation	Complexity
Insert	O(log n)
Delete Highest Priority	O(log n)
Access Highest Priority	O(1)

Advantages

- Automatic priority handling
- Efficient emergency management
- Simple implementation

3.3.4 List

Purpose

A **list** is used to maintain the donation history.

list<DonationRecord> donationHistory;

Why List?

Donation records are stored in the order they occur. A list allows easy insertion of new records without affecting the existing order.

Operations Performed

- Store donation records
- Display donation history

Time Complexity

Operation	Complexity
Insert	O(1)
Delete	O(1)
Traversal	O(n)

Advantages

- Easy insertion
- Maintains sequence of records
- Suitable for historical data

3.4 Object-Oriented Programming Concepts Used

The project is developed using the principles of Object-Oriented Programming to improve code organization and maintainability.

Classes Used

- Donor
- BloodRequest
- DonationRecord
- DonorManager
- RequestManager
- FileManager

OOP Concepts Applied

- Encapsulation
- Constructors
- Getters and Setters
- Objects
- Member Functions

The use of OOP makes the code modular, reusable, and easier to understand.

3.5 File Handling

The project uses **fstream** to store data permanently.

Files Used

- donors.txt – Stores donor information.
- requests.txt – Stores blood request details.
- history.txt – Stores donation history.

When the application starts, the data is loaded automatically from these files. Whenever the user saves the project data or exits the application, the latest information is written back to the files. This ensures that donor records and request information are preserved even after the application is closed.

3.6 Important Source Code

Instead of attaching every source file, only include the most important code snippets with brief explanations.

Code Snippet 1 – Donor Class

class Donor

```
{
private:
    string donorID;
    string name;
    string bloodGroup;
    bool available;

public:
    void input();
    void display();
};
```

Explanation:
This class stores all information related to a blood donor. Each donor object contains personal details and member functions to input and display donor information.

Code Snippet 2 – Register Donor

```
void DonorManager::registerDonor()
{
    Donor donor;
    donor.input();
    donors.push_back(donor);
}
```

Explanation:
This function creates a new donor object, collects the donor's details from the user, and stores the object in the vector containing all donor records.

Code Snippet 3 – Blood Group Search

```
bloodIndex[bloodGroup].push_back(index);
```

Explanation:
This statement stores the donor index in an unordered_map based on the donor's blood group. It enables faster searching by grouping donors according to blood type.

Code Snippet 4 – Emergency Request

```
priority_queue<BloodRequest> emergencyQueue;
```

Explanation:
A priority queue stores emergency blood requests. Requests with higher priority are processed before normal requests, ensuring quicker response during emergencies.

Code Snippet 5 – File Saving

```
ofstream file("donors.txt");
```

Explanation:
This statement creates or opens the donor data file and writes donor information to permanent storage using file handling.

CHAPTER 4

Screenshots of the Project

4.1 Donor Registration

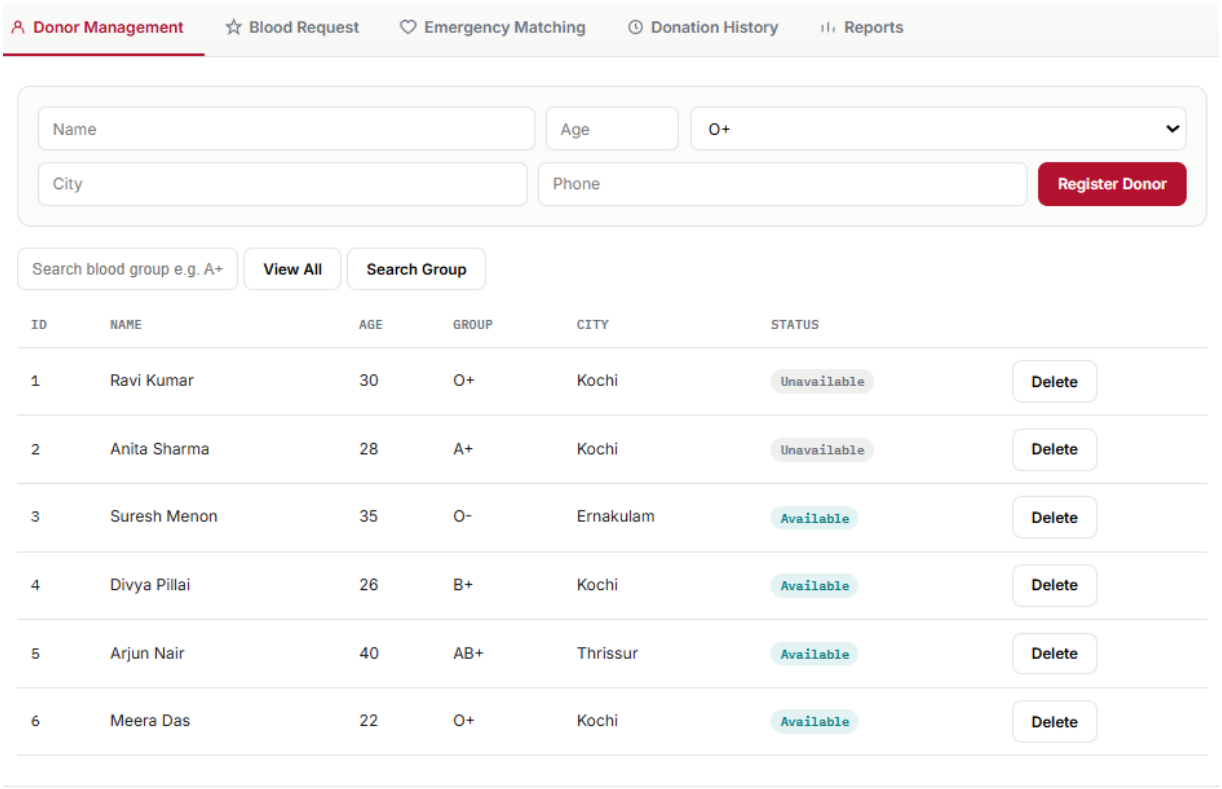


Figure 4.1: Donor Registration

This screen allows the user to register a new blood donor by entering details such as Donor ID, Name, Age, Gender, Blood Group, Contact Number, City, and Availability Status. Once the information is entered, the donor is successfully added to the system. This screen also displays the complete list of registered blood donors stored in the system. The information includes donor ID, name, blood group, contact details, city, and availability status. It helps users quickly view all available donor records. The Search Donor screen enables users to search for donors based on a specific blood group. The system retrieves and displays all donors matching the entered blood group. This functionality uses an unordered_map to improve search efficiency. This screen allows the user to update existing donor information or remove donor records from the system. Users can modify details such as phone number, city, or availability status whenever required.

Purpose:

- Add new donor information.
- Store donor records for future use.
- Display all donor information.
- Verify stored donor records.

- Quickly locate suitable donors.
- Reduce search time during emergencies.
- Keep donor information updated.
- Remove inactive or incorrect records.

4.2 Blood Request Management

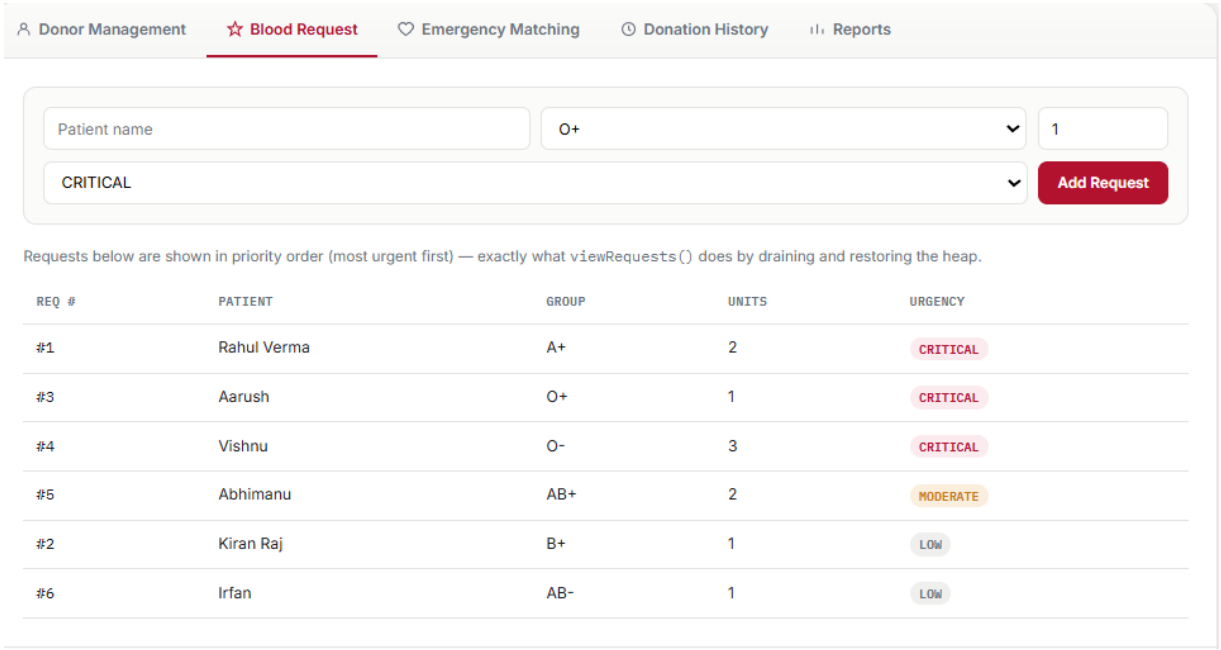


Figure 4.2: Blood Request Management

Description

This interface is used to register new blood requests from hospitals or patients. The user enters the patient's name, required blood group, quantity of blood units, and the priority level. All requests are stored for further processing.

Purpose:

- Record blood requests.
- Maintain request information.

4.3 Emergency Matching

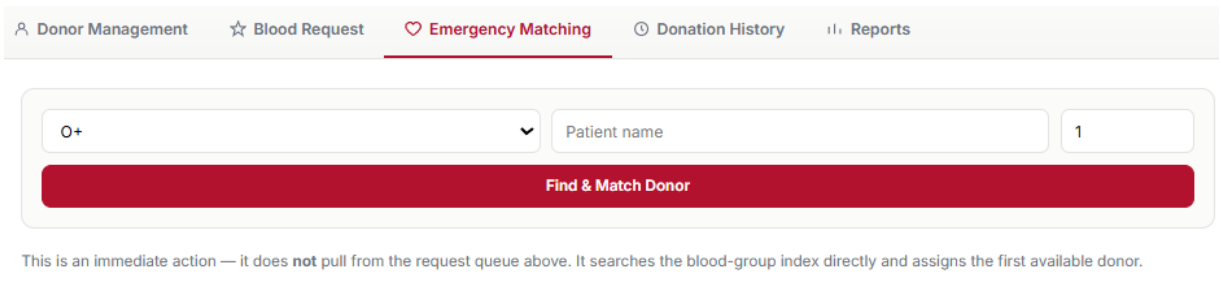


Figure 4.3: Emergency Matching

Description

This module searches for available donors matching the requested blood group. Emergency requests are handled using a priority queue so that urgent cases are processed before normal requests. Matching donor details are displayed on the screen.

Purpose:

- Find suitable donors quickly.
- Process emergency cases efficiently.

4.4 Donation History

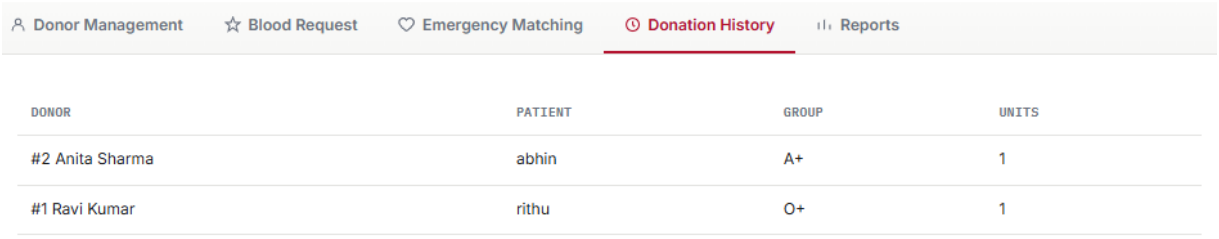


Figure 4.4: Donation History

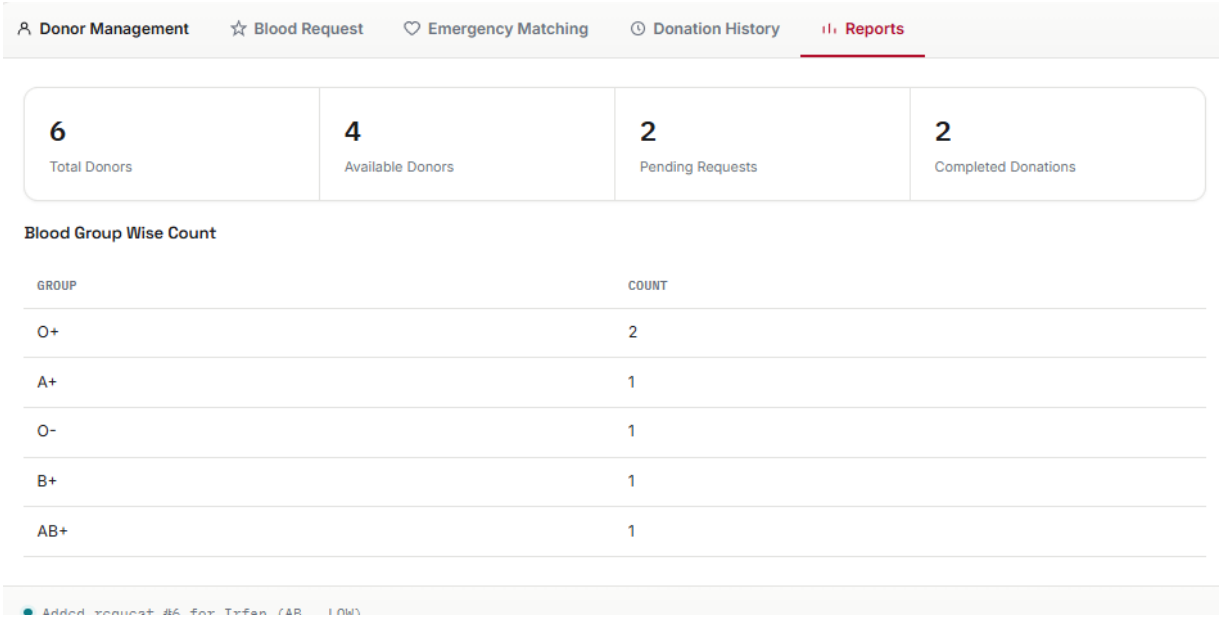
Description

This screen displays the history of successful blood donations recorded in the system. It includes donor information, recipient details, blood group, and donation date. The records are maintained in chronological order using a list.

Purpose:

- Maintain donation records.
- Track completed donations.

4.5 Reports



Description

The Reports module provides summarized information about the system. It displays the total number of registered donors, available donors, and the number of donors in each blood group. These reports help users understand the overall status of the donor database.

Purpose:

- Display project statistics.
- Analyze donor availability.

CHAPTER 5

Future Scope of the Project

The Smart Blood Donation Emergency Matching System has been developed as a standalone console-based application to demonstrate the practical implementation of Data Structures and Object-Oriented Programming concepts using C++. Although the current system successfully manages donor information, blood requests, emergency matching, donation history, and report generation, there are several opportunities for future enhancements that can improve its functionality and usability.

One possible enhancement is the development of a Graphical User Interface (GUI). The current project uses a console-based interface, which is simple and suitable for academic purposes. In the future, the application can be converted into a desktop application using frameworks such as Qt or C++ Builder, making it more user-friendly and visually appealing.

Another improvement would be replacing the text-file storage with a database management system such as MySQL or SQLite. A database would provide better security, faster data retrieval, and improved management of large amounts of donor and request information.

The current project performs donor matching based on exact blood group matching. In future versions, the system can be enhanced by implementing blood compatibility rules, allowing the application to suggest compatible donors even when an exact blood group match is unavailable. This would make the system more useful in real emergency situations.

The project can also be expanded by adding a hospital and blood bank management module. This feature would allow hospitals and blood banks to register, update blood availability, and manage requests from multiple locations within a single system.

Another useful enhancement is the addition of donor eligibility tracking. The system could automatically calculate the waiting period between blood donations and notify users when they become eligible to donate again.

The application may also include SMS or email notification services to inform registered donors whenever an emergency blood request matching their blood group is generated. This feature would improve communication and help reduce the response time during emergencies.

Finally, the project can be developed into a mobile or web-based application so that donors and hospitals can access the system remotely. Such an enhancement would increase accessibility and allow faster communication during critical situations.

Overall, these enhancements can transform the current academic project into a more comprehensive blood donation management system suitable for practical use in hospitals and blood banks.

CHAPTER 6

Conclusion

Conclusion

The **Smart Blood Donation Emergency Matching System** was successfully developed as a standalone console-based application using **C++** to demonstrate the practical implementation of **Data Structures**, **Object-Oriented Programming**, and **File Handling** concepts. The project addresses the real-world problem of efficiently managing blood donor information and assisting in the quick identification of suitable donors during emergency situations.

The application provides essential functionalities such as donor registration, donor search, blood request management, emergency donor matching, donation history management, and report generation. These features make the system organized, easy to use, and capable of managing blood donation records in a structured manner. The use of appropriate data structures, including **vector**, **unordered_map**, **priority_queue**, and **list**, improves the efficiency of storing, searching, and managing information while also demonstrating their practical applications in software development.

This project also helped strengthen the understanding of important programming concepts such as classes, objects, encapsulation, modular programming, file handling, and the Standard Template Library (STL). Developing the project from planning to implementation improved problem-solving abilities, debugging skills, and the understanding of how Data Structures can be applied to solve real-life problems.

Although the project has been developed primarily for academic purposes, it provides a strong foundation for future enhancements such as database integration, graphical user interfaces, blood compatibility checking, and hospital management integration. These improvements can make the application more suitable for practical deployment in healthcare environments.

In conclusion, the project successfully achieved its objectives by combining theoretical knowledge with practical implementation. It demonstrates that Data Structures and Algorithms can be effectively used to build efficient and meaningful applications while contributing to the solution of an important social problem.

Learning Outcomes

Through the development of this project, the following learning outcomes were achieved:

Gained practical experience in developing a complete C++ application.

Understood the implementation of Object-Oriented Programming concepts.

Applied Data Structures such as **vector**, **unordered_map**, **priority_queue**, and **list** in solving real-world problems.

Learned to implement file handling for permanent data storage.

Improved programming, debugging, and testing skills.

Enhanced problem-solving and logical thinking abilities.

Understood the importance of selecting appropriate data structures to improve application efficiency.

Experienced the complete software development process, including planning, implementation, testing, documentation, and presentation.